

Скрипты

Скрипт – это инструмент, позволяющий разово выполнить произвольный код разработчика в контексте существующего сервиса.

Файл скрипта – это архив, содержащий модуль, в составе которого находится справочник объектов [.orgx](#) с описанием метода скрипта, внедряемый на сервис приложения. Внедрение скрипта происходит через сервис [Хоттабыч](#).

Описание процесса вызова скрипта

Хоттабыч разворачивает копию целевого сервиса или сервис с минимальным набором модулей на одной из бизнес-логик целевого сервиса. В развернутый сервис внедряется модуль разработчика. Клиенты персональных сервисов разбиваются на группы, производится вызов служебного метода **DeveloperScript.ExecuteAll** с помощью [сервиса распределённых вычислений \(DWC\)](#). Внутри происходит вызов методов скрипта разработчика последовательно в лексикографическом порядке названий методов. В случае персонального сервиса вызов методов выполняется с использованием [TenantContext](#) под каждым клиентом приложения.

[Для разработчиков] Инструкция по созданию скрипта

Шаг 1: Подготовить скрипт

1. Создайте через [Genie](#) новый [справочник объектов](#) (*.orgx). Использование Genie – гарантия валидности файла.
2. Внутри справочника создайте новый объект БЛ с именем DeveloperScript. Имя объекта строго фиксировано и не подлежит изменению.
3. В объекте БЛ создайте новый метод БЛ типа "Custom method".
 - Соблюдайте [правила именования](#).
 - Реализовать метод можно любым способом: SQL или Python.
 - Сигнатура метода не должны иметь входных параметров.
4. В методе реализуйте скрипт.



Какие механизмы разрешено использовать в методе?

В реализации метода вы можете делать внутренние вызовы методов своего сервиса и внешние вызовы других сервисов, использовать любые механизмы платформы, которые обычно используются в методах бизнес-логики (см. [Взаимодействие между сервисами](#) и [Сервер приложения](#)).

Полученный справочник объектов – минимально необходимое содержимое скрипта.

Требования к сервису-исполнителю скрипта

Для выполнения скрипта сервис должен удовлетворять следующим требованиям:

1. Реализован на платформе Saby
2. Имеет базу данных
3. В режиме запуска "копия сервиса" требуется, чтобы сервис имел модули бизнес-логики:
 - Python Core
 - Cloud Interaction-Py
 - Redis Client-Py
 - RequestBrokerClient
 - HugePayload

Эти модули гарантированно есть в сервисе, если сервис унаследован от basic_platform.s3srv.

Если модулей в сервисе нет, то выполнение скрипта возможно только в режиме "минимальный сервис".

Правила именования метода и его реализация

Название метода должно быть осмысленным, соответствовать [стандартам именования](#), а также уникальным в рамках одного исполнения скриптов. Исходите из того, что за одно исполнение скриптов могут применяться скрипты, написанные разными разработчиками.

Модуль как дополнение к скрипту

Если есть файлы s3mod в качестве дополнительных ресурсов для выполнения скрипта, то название модуля (внутри файла в теге bl_module атрибут name) должно быть в формате DeveloperScript_<Уникальное имя>.

Наличие одинаково названных модулей в сервисе приводит к тому, что загружаются зависимости только одного из модулей. Если вы не создадите s3mod сами, он будет сгенерирован автоматически, при чем в нем уже будет зависимость Python Core, которая позволит импортировать содержащиеся в модуле python файлы.

Шаг 2: Создать архив

Упакуйте *.orgx и остальные объекты в zip-архив. Файлы *.orgx и s3mod (если есть) должны быть в корне архива, остальные файлы должны располагаться относительно них. Имя архива указывается согласно [правилам](#).

Пример архива: [s_familiao_1174868331_v2.zip](#)

Для неперсональных приложений необходимо также создать файл со списком сервисов, на которых необходимо выполнить скрипт. Для всех стендов файл является общим и именуется "services".

Пример архива с файлом services: [s_familiao_1174868332_v2.zip](#)



Используйте [7-Zip](#) для создания архива, так как при использовании стандартных средств на Windows процесс создания может зависать.

Для CentOS нажмите правой кнопкой мыши на файл или папку и выберите "Сжать" в контекстном меню.

Состав архива

В архиве обязательно должен быть хотя бы один *.orgx файл (см. [справочник объектов](#)).

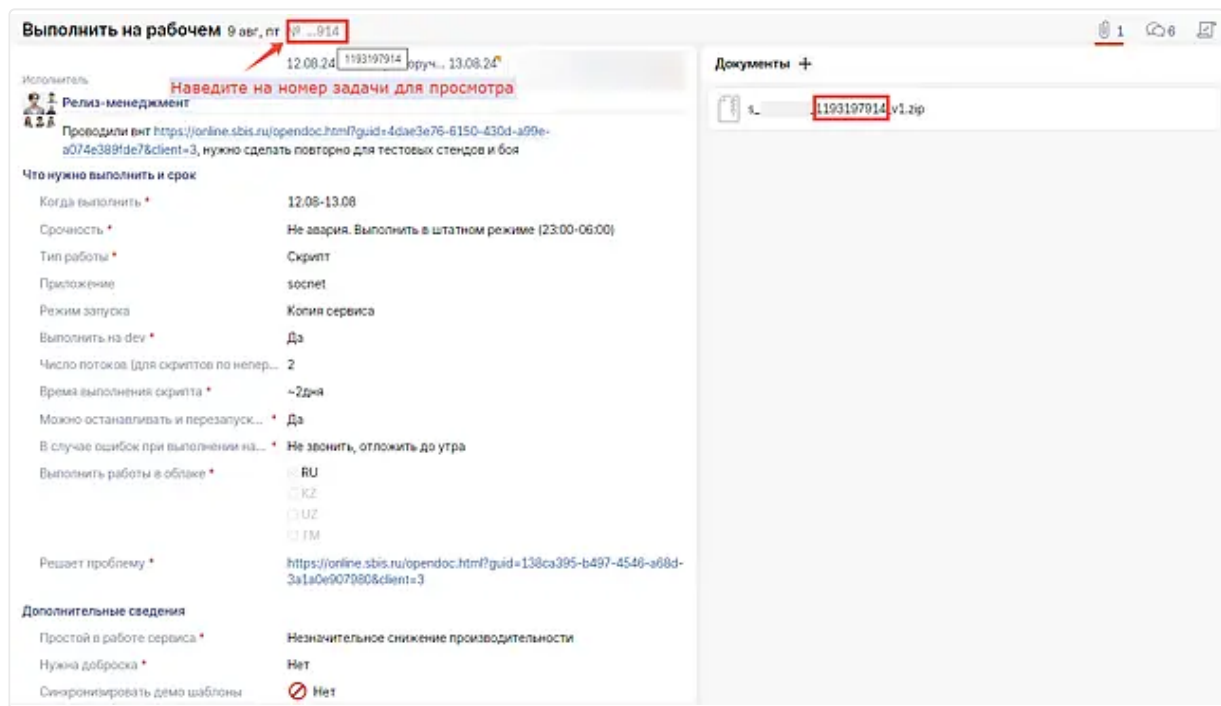
Опционально в состав архива можно включить любые файлы, необходимые для исполнения скриптов, например, описание модуля s3mod для подключения нужных модулей или python-файлы, подключаемые в основном методе.

Расширение архива и кодировка имён вложенных файлов

- *.zip
- Допустимая кодировка имён вложенных файлов архива – CP866.

Шаг 3: Заявить выполнение скрипта

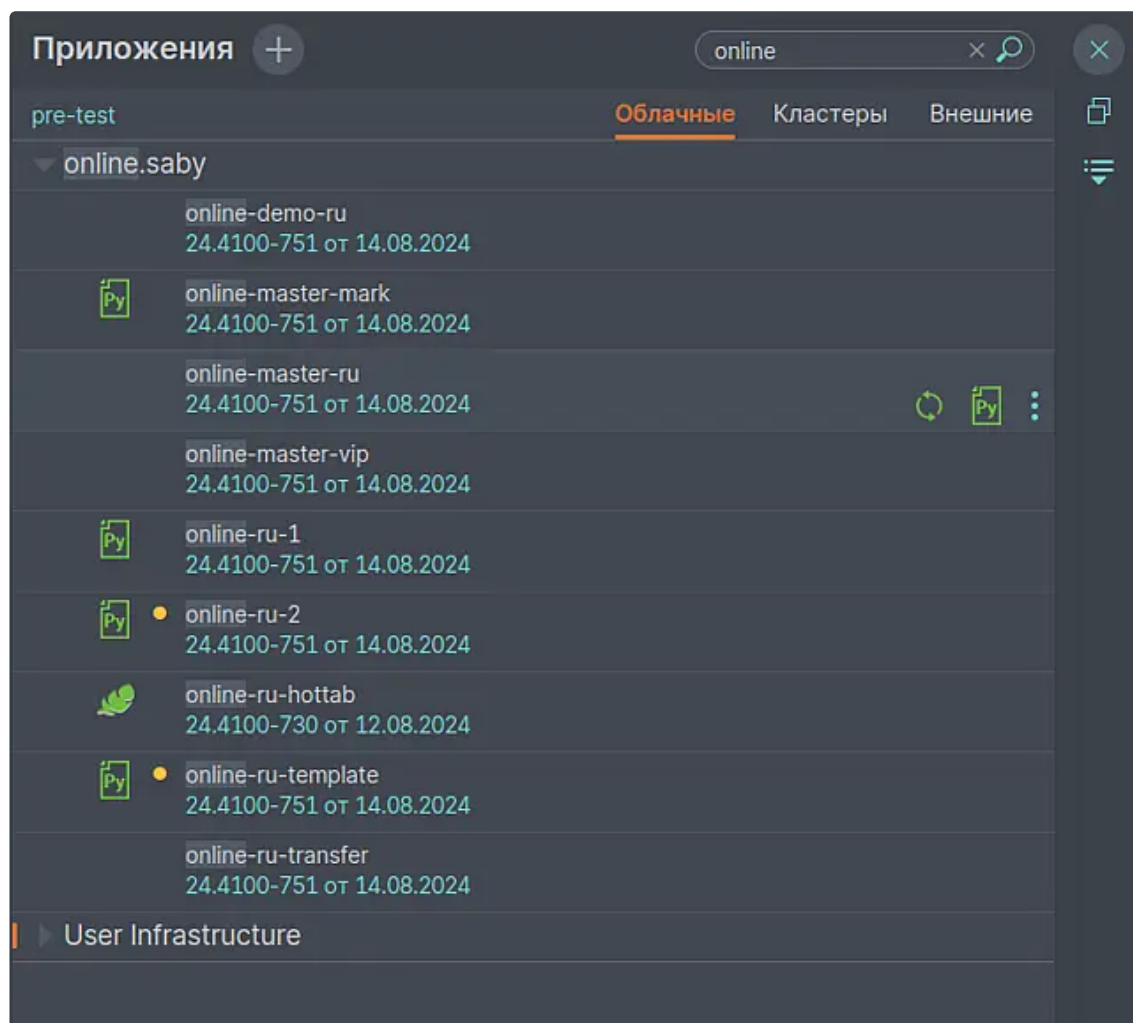
1. На online.sbis.ru создайте задачу с типом "Выполнить на рабочем" (ВНР), согласно [регламенту](#).
2. Переименуйте архив скрипта, используя номер задачи.



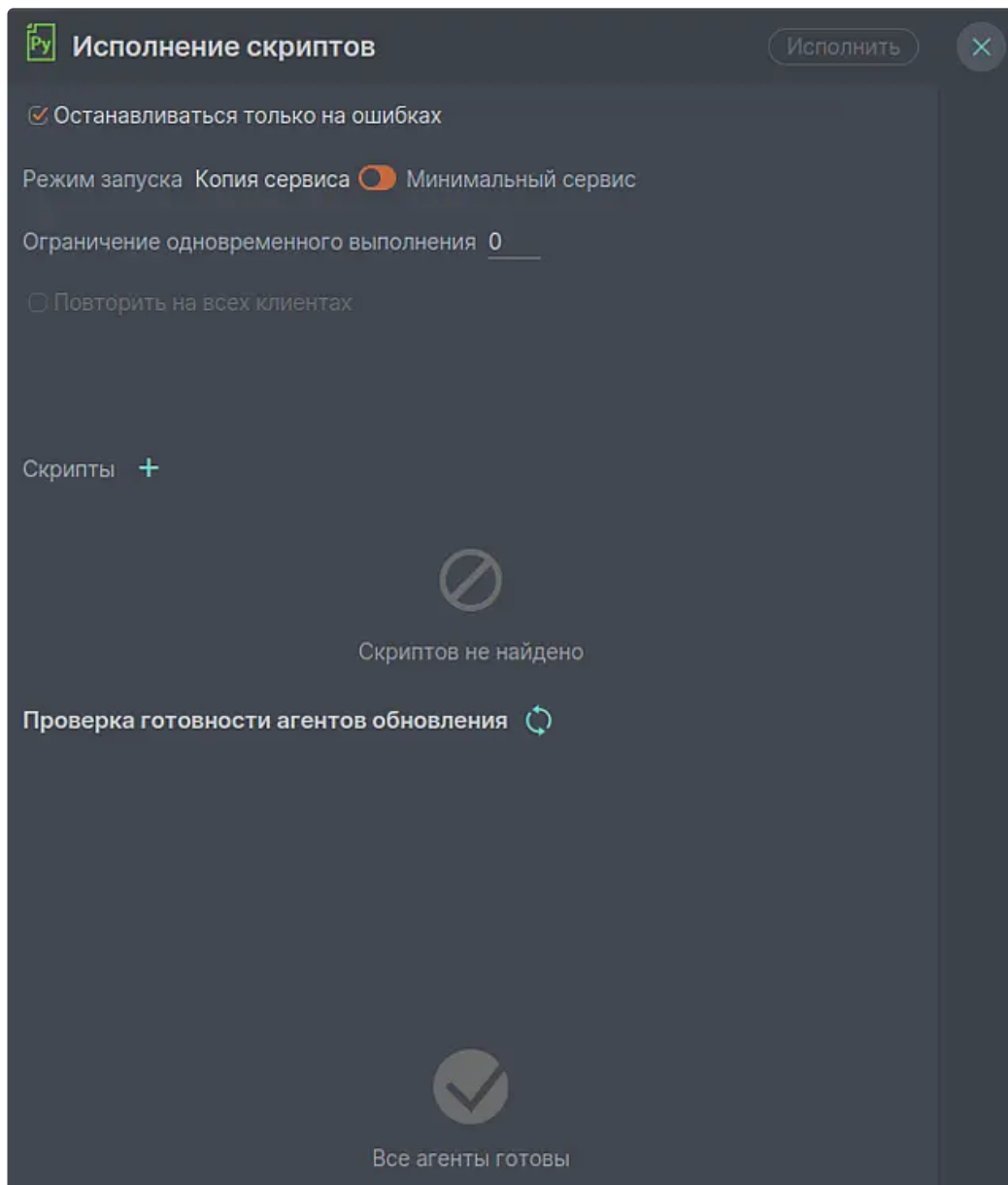
3. Прикрепите архив к ВНР.

[Для эксплуатации] Инструкция по внедрению скрипта на сервис и его выполнение

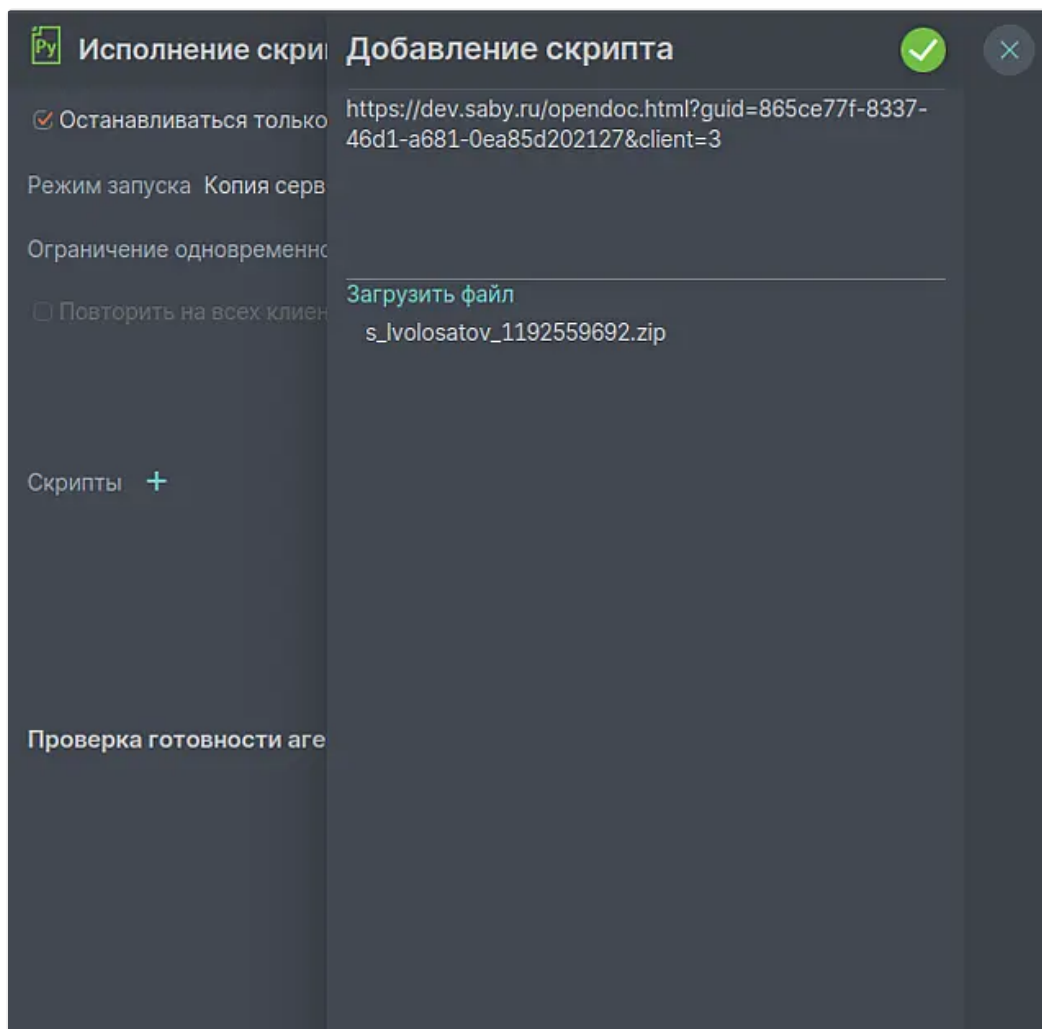
1. Выберите приложение и нажмите значок



2. Нажмите кнопку "+".



3. Загрузите [архив скрипта](#) и укажите ссылку на VNP.



Настройки запуска скрипта:

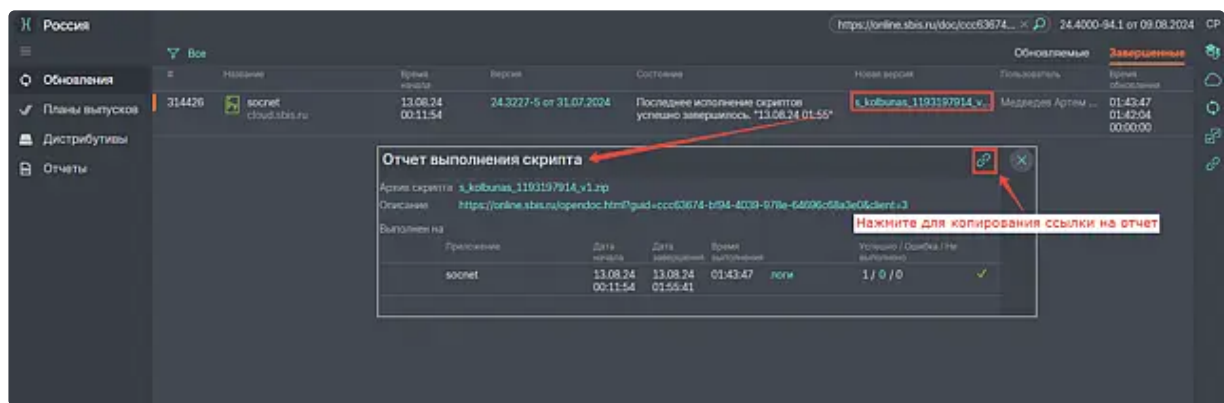
- **Останавливаться только на ошибках.** Признак непрерывного выполнения, процесс не будет останавливаться при завершении фаз подготовки и обновления. По умолчанию включен.
- **Режим запуска.** Выбор способа развертывания скриптового сервиса: копия или минимальный сервис. По умолчанию разворачивается полная копия целевого сервиса. Если в ВНР указано, что скрипт нужно выполнить на минимальном сервисе, то нужно выбрать режим минимального сервиса. В этом случае будет развернут сервис с минимальным набором модулей для выполнения скрипта.
- **Ограничение одновременного выполнения.** Соответствует полю ВНР ["Ограничение одновременного выполнения"](#). Поле отображается только при запуске скрипта на персональном приложении. Ограничивает максимальное число клиентов, одновременно выполняющих скрипт. Значение по умолчанию – 0, означает, что скрипт будет выполняться без ограничений.
- **Повторять на всех клиентах.** Используется только при запуске скрипта на персональном приложении. Доступен только, если выбранный скрипт уже выполняется на выбранном приложении, и был выполнен не на всех клиентах.
- **Выбор скрипта.** Выбор одного из уже загруженных скриптов ранее. Новый скрипт можно загрузить по кнопке "+". Загруженные скрипты хранятся в текущей сборке дистрибутива приложения в сервисе "Хранилище дистрибутивов", просмотр которого можно осуществить в разделе "Репозитории".

Отчет выполнения скрипта

Скрипт, особенно когда он должен быть исполнен на онлайн, может запускаться на разных приложениях, причем не по одному разу, в случае его прерывания. Информация о выполнении скрипта отображается в отчет, который также позволяет получить доступ к файловым и облачным логам скрипта.

Отчет выполнения скрипта расположен в реестре **Обновления**. Введите в строке поиска ссылку на [ВНР](#) и нажмите на содержимое колонки **Новая версия**. Карточка отчета содержит архив скрипта, ссылку на ВНР и информацию о результатах выполнения скрипта на приложениях.

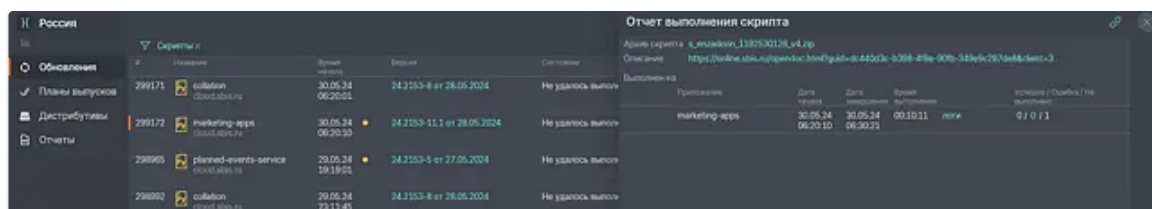
Отчет можно передать в виде ссылки с помощью значка "Копировать в буфер обмена". Ссылка открывается для всех пользователей, имеющих минимальные права доступа.



В зависимости от приложения, на котором выполняется скрипт, информация о результатах выполнения может отличаться.

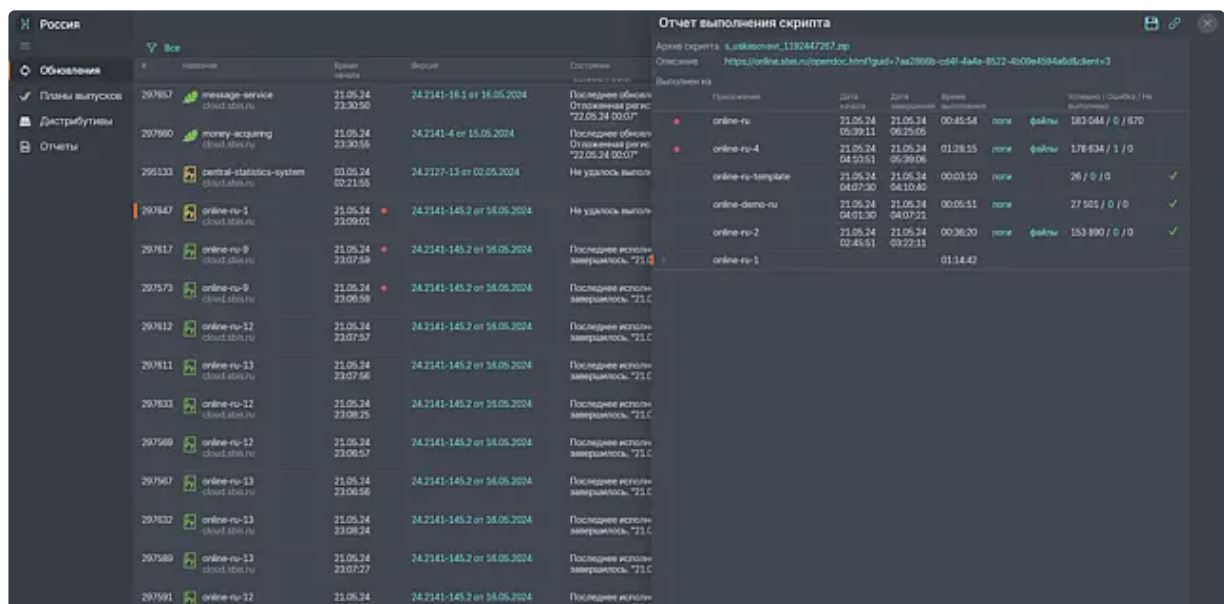
При выполнении скрипта по прочему сервису отображается только одно из состояний:

- **Успешно** – успешное выполнение скрипта на клиенте или сервисе
- **Ошибка** – скрипт некоторое время выполнялся, а затем прервал работу из-за ошибки. Возможна ситуация, когда скрипт прерывается, из-за того, что он не был выполнен за определенное время.
- **Не выполнено** – ошибок нет, но скрипт не выполнен. Это означает, что скрипт выполнялся и его прервали до завершения.



При выполнении по онлайн скрипты выполняются на клиентах, распределенных по всем приложениям. В отчете можно увидеть сколько клиентов приложения обработано в скрипте успешно, сколько упали с ошибкой и сколько не выполнено из-за прерывания до завершения.

Например, для приложения online-ru выполняли скрипт на 183044 клиентах, а на 670 – выполнение скрипта было прервано.



Также в отчете отображаются ссылки на [ЛОГИ ОБНОВЛЕНИЯ](#) и [ФАЙЛЫ](#), созданные в процессе работы скрипта. Файлы можно скачать одним архивом с помощью значка "Сохранить" или по отдельности для каждого приложения.

И	Название	Время начала	Время	Состояние	Выполнен на	Детали	Результат
299114	online-master-ru	30.05.24 03:32:18	24.2153-105.1 от 28.05.2024	Завершено	online-ru-6	30.05.24 03:32:18	194 418 / 0 / 0
299002	online-ru-10	30.05.24 03:06:04	24.2153-105.1 от 28.05.2024	Не удалось выполнить	online-ru-5	30.05.24 03:22:45	186 478 / 0 / 0
299132	online-ru-partners	30.05.24 03:32:13	24.2153-105.1 от 28.05.2024	Последнее исполнение завершилось: "301"	online-ru-11	30.05.24 03:21:44	174 255 / 0 / 0
299127	online-ru-template	30.05.24 03:32:10	24.2153-105.1 от 28.05.2024	Последнее исполнение завершилось: "301"	online-ru-7	30.05.24 03:21:43	131 579 / 0 / 0
299140	online-ru-account-ru	30.05.24 03:32:17	24.2153-105.1 от 28.05.2024	Последнее исполнение завершилось: "301"	online-ru-10	30.05.24 03:21:42	187 B
299138	online-ru-vip2	30.05.24 03:32:14	24.2153-105.1 от 28.05.2024	Последнее исполнение завершилось: "301"	online-ru-6	30.05.24 03:21:40	179 572 / 0 / 0
299141	online-ru-0	30.05.24 03:32:18	24.2153-105.1 от 28.05.2024	Последнее исполнение завершилось: "301"	online-ru-0	30.05.24 03:21:55	29 / 0 / 0
299123	online-ru-7	30.05.24 03:32:12	24.2153-105.1 от 28.05.2024	Последнее исполнение завершилось: "301"	online-ru-account-ru	30.05.24 03:21:40	13 / 0 / 0
299126	online-ru-8	30.05.24 03:32:15	24.2153-105.1 от 28.05.2024	Последнее исполнение завершилось: "301"	online-ru-12	30.05.24 03:21:39	200 002 / 0 / 0
299114	online-ru-9	30.05.24 03:32:17	24.2153-105.1 от 28.05.2024	Последнее исполнение завершилось: "301"	online-ru-vip	30.05.24 03:21:39	82 / 0 / 0
299117	online-ru-2	30.05.24 03:32:16	24.2153-105.1 от 28.05.2024	Последнее исполнение завершилось: "301"	online-ru-13	30.05.24 03:21:38	190 765 / 0 / 0
					online-ru-9	30.05.24 03:21:37	172 547 / 0 / 0
					online-master-ru	30.05.24 03:21:37	1 / 0 / 0

Анализ выполнения скриптов

Логирирование скриптов соответствует текущим [настройкам логирирования](#) целевого сервиса и метода `DeveloperScript.ExecuteAll` (выполнение скрипта логируется под этим методом, так как методы скрипта разработчика делаются внутренними вызовами).

Для просмотра логов выполнения скриптов необходимо в сервисе логов применить фильтр по целевому сервису и методу `DeveloperScript.ExecuteAll`.

Вызов каждого метода скрипта в логах обрамляется сообщениями:

- `[script][start][DeveloperScript.НазваниеМетода][идентификатор клиента в облаке]`
- `[script][finish][DeveloperScript.НазваниеМетода][идентификатор клиента в облаке]`

С их помощью можно найти логи по конкретному методу.



Рекомендации

Не рекомендуется использовать сервис логов для записи результатов выполнения скрипта, так как

- логи могут быть очищены раньше, чем удастся забрать их;
- результат может быть не залогирован, если метод выполнялся быстрее порога логирирования.

При выполнении скрипта в Хоттабыче, на фазе очистки осуществляется проверка асинхронных очередей, т.е. проверяется, что на сервисе, на котором выполнялся скрипт, отсутствуют (успели завершиться) выполняемые скриптовые методы.

Эти проверки отражены в журнале скриптового узла на фазе очистки и начинаются с записи "Запускаем проверку активности скриптового сервиса".

338055 online- Журнал сообщений обновления через Docker		dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/ (системное имя: 'dev-script-online-107634')		
Подготовка выполн		Найти		
Версия: 24.6		Все		
Статус:				
Дата	Статус	Сообщение		
02.12.24 03:12:48:816	Обновление выполняется	Попытка запроса версии по адресу: http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/?version		
02.12.24 03:12:48:860	Обновление выполняется	Версия контейнера совпадает с версией дистрибутива ("24.6102-28")		
02.12.24 03:12:48:921	Обновление завершено	Контейнер готов к работе		
02.12.24 03:32:33:862	Очистка ожидается	Задание 'stop' для контейнера успешно сформировано.		
02.12.24 03:32:39:542	Очистка выполняется	Запускаем проверку активности скриптового сервиса.		
02.12.24 03:32:39:571	Очистка выполняется	http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/?load&srv=1 -> BusyWorkers:5 FreeWorkers:1 ExtraWorkers:0 QueueSize:1 ProcessedRequests:18843 Errors:0 MaxWaitTime:10.		
02.12.24 03:32:39:583	Очистка выполняется	Обнаружено 5 занятых воркеров (Сервис: http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/).		
02.12.24 03:32:39:595	Очистка выполняется	http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/?queue&srv=1 -> {"d":		
02.12.24 03:32:39:606	Очистка выполняется	В синхронной очереди 1 запросов (Сервис: http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/).		
02.12.24 03:32:40:574	Очистка выполняется	http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/?load&srv=1 ->		
02.12.24 03:32:40:590	Очистка выполняется	Обнаружено 5 занятых воркеров (Сервис: http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/).		
02.12.24 03:32:40:604	Очистка выполняется	http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/?queue&srv=1 -> {"d":		
02.12.24 03:32:40:617	Очистка выполняется	В синхронной очереди 1 запросов (Сервис: http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/).		

В случае, если проверка показывает, что все воркеры свободны (методы не выполняются), осуществляется остановка скриптового узла и завершение обновления.

Однако возможны ситуации, когда асинхронная очередь по той или иной причине не пуста. В этом случае, по истечении таймаута проверки, обновление (выполнение скрипта) останавливается Хоттабычем, а в журнале скриптового узла фиксируется соответствующая запись,

338055 online- Журнал сообщений обновления через Docker				
dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/ (системное имя: 'dev-script-online-107634')				
Версия:	24.6	Найти		
Статус:		Все		
Дата	Статус	Сообщение		
02.12.24 03:35:01:945	Очистка выполняется	Обнаружено 5 занятых воркеров (Сервис: http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/).		
02.12.24 03:35:01:968	Очистка выполняется	http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/?queue&srv=1 -> ("d":		
02.12.24 03:35:01:991	Очистка выполняется	В синхронной очереди 1 запросов (Сервис: http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/).		
02.12.24 03:35:02:016	Очистка выполняется	Обнаружено, что в данный момент исполняются методы (Сервис: http://dev-script-online-107634.online-ru-		
02.12.24 03:35:20:583	Очистка выполняется	http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/?load&srv=1 ->		
02.12.24 03:35:20:599	Очистка выполняется	Обнаружено 5 занятых воркеров (Сервис: http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/).		
02.12.24 03:35:20:616	Очистка выполняется	http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/?queue&srv=1 -> ("d":		
02.12.24 03:35:20:632	Очистка выполняется	В синхронной очереди 1 запросов (Сервис: http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/).		
02.12.24 03:35:20:646	Очистка выполняется	Обнаружено, что в данный момент исполняются методы (Сервис: http://dev-script-online-107634.online-ru-		
02.12.24 03:35:20:679	Очистка завершена	На сервисе 'http://dev-script-online-107634.online-ru-11.svc.prod1.k8s.tensor.ru:8000/service/' обнаружены занятые воркеры, поэтому его остановка прервана. Вероятнее всего это последствия выполнения скрипта. Нужно возобновить скрипт после прекращения активности на сервисе.		

Эта запись означает, что необходимо разобраться с очередью (дождаться завершения методов, очистить очередь в облаке вручную и т.д., в зависимости от ситуации) и после этого **возобновить** (кнопка "Возобновить") обновление в Хоттабыче.

После возобновления будет повторно проверена очередь и, если она пуста, обновление будет завершено штатно.

При остановке обновления из-за наличия такой очереди **запрещается завершать обновление кнопкой "Принудительно завершить"**, т.к. это приведёт к завершению обновления в Хоттабыче, но поднятые скриптовые узлы не будут остановлены.

Сохранение результатов выполнения скрипта

Для сохранения логов и данных, полученных в процессе выполнения скрипта, с возможностью скачивания после выполнения предлагается два варианта:

1. Сохранение в Redis, подходит для сбора данных небольшого размера (не более 5 Мб на одного клиента) по клиентам основного сервиса (online/inside). Данные сохраняются в разрезе приложения, т.е. на одно приложение будет один файл.
2. Сохранение файловых логов на бизнес-логику сервиса подходит для прочих сервисов, доступный объем ограничен размерами диска бизнес-логики. Данные сохраняются в разрезе сервиса/персональной группы, т.е. на каждые сервис/группу будет один архив с файлами.

Сохранение данных в Redis

`sbis.DeveloperScriptAPI.LogRedis(Data: sbis.Record)` переданный record преобразуется в json и записывается в хэш-таблицу Redis с ключом в виде идентификатора клиента. При повторной записи по этому же клиенту, данные будут перезаписаны. После завершения скрипта, хэш-таблица записывается в json-файл и сохраняется в хранилище файлов. Скачать файлы можно из отчета скрипта.

Сохранение файловых логов на бизнес-логиках сервиса

Для сохранения файлов в процессе выполнения скрипта доступны методы: `sbis.DeveloperScriptAPI.LogMsg(Message: str)`

Логи записываются в csv файл, формат записи логов:

- для персонального сервиса: [дата и время, идентификатор клиента в облаке, сообщение]
- для неперсонального: [дата и время, сообщение].

`sbis.DeveloperScriptAPI.LogArrayMsg(Messages: [str])`

Логи записываются в csv файл. Один вызов метода записывает одну строку в файл. Каждый элемент записывается в отдельную ячейку.

При использовании методов `DeveloperScriptAPI.LogMsg` и `DeveloperScriptAPI.LogArrayMsg` размер и число файлов csv определяются параметрами облака:

Управление версиями.Скрипты.МаксимальныйРазмерЛога = 100 Мб

При достижении этого размера, будет создан еще один файл, запись продолжится туда.

Управление версиями.Скрипты.МаксимальноеЧислоЛогов = 20

Для получения директории с файловыми логами доступен метод `sbis.DeveloperScriptAPI.LogsDirectory()` -> `str`.

Метод возвращает путь до папки с логами. Произвольные файлы, созданные разработчиком скрипта в этой директории будут собраны и доступны для скачивания после завершения работы скрипта.

Все файловые логи пишутся на БЛ, выполняющую скрипт. После выполнения скрипта файлы упаковываются в архив, загружаются в хранилище файлов. Скачать файлы можно из отчета скрипта.

Время доступности файлов определяется параметром облака:

Управление версиями.Логи.МинимальныйПериодХранения = 14 дней



Рекомендации

Файлы каждой БЛ, участвующей в выполнении скрипта, упаковываются в отдельный архив, название которого содержит имя хоста БЛ и системное имя сервиса.



Ограничения

Методы доступны только в контексте выполнения скрипта. Хоттабычем, поэтому локально работу не проверить.

Для персональных сервисов для каждой персональной группы серверов скрипт выполняется на собственной бизнес-логике, поэтому для каждой бизнес-логики будет свой архив. Большое число файлов затруднит анализ.

Типичные задачи и способы их решения

Выполнить скрипт на конкретных клиентах

Для того чтобы выполнить скрипт на конкретных клиентах, нужно в корень архива скрипта добавить файлы, содержащие идентификаторы клиентов. Можно указывать клиентов как облака (ID в облаке), так и биллинга (Номер аккаунта). Для каждого стенда можно указать свой набор клиентов для выполнения скрипта.



Формат файла

Файл не должен иметь расширения. Если набор клиентов не указан или файл имеет неправильный формат, скрипт будет выполнен на всех клиентах продукта.

Формат названия файла:

СистемноеИмяОблака_ТипИДКлиентов_clients (СистемноеИмяОблака можно узнать в реестре **Настройки** → **Облака** → столбец **Системное имя**), например:

fix_cloud_clients	клиенты облака для фикса
fix-kz_cloud_clients	клиенты облака в Казахстане для фикса

pre-test_billing_clients	клиенты биллинга для пре-теста
rc_cloud_clients	клиенты облака для rc
prod_cloud_clients	клиенты облака для боя
prod_billing_clients	клиенты биллинга для боя
prod-kz_cloud_clients	клиенты облака в Казахстане для боя
prod-uz_billing_clients	клиенты биллинга в Узбекистане для боя

Внутри файла должны содержаться только числа (идентификаторы), разделенные пробелами, знаками табуляции или переносами строки. Перед выполнением скрипта номера аккаунтов будут сконвертированы в ID клиентов облака. Скрипт может содержать несколько разных файлов клиентов для разных стендов, поэтому можно один и тот же архив со скриптом использовать на разных стендах, будет использован набор клиентов для того стенда, на котором скрипт выполняется.

Пример скрипта:

```

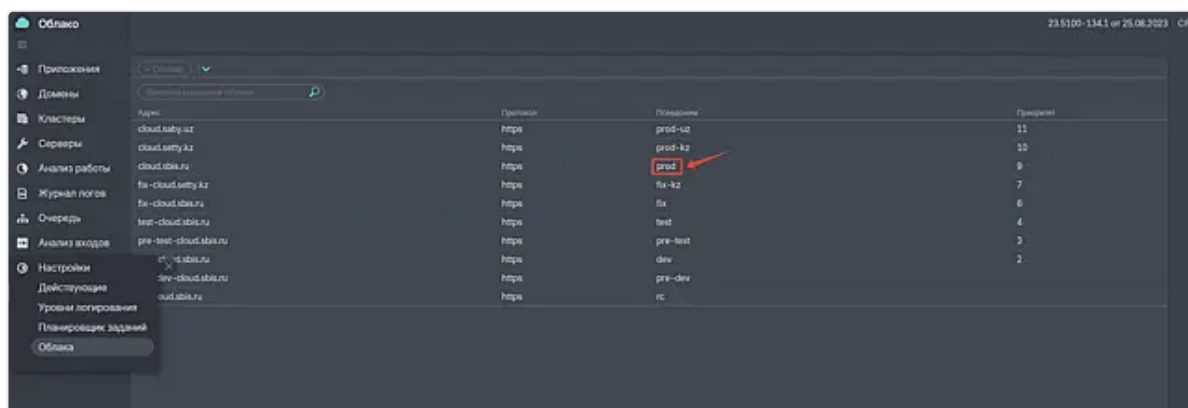
1  <?xml version="1.0" encoding="UTF-8" ?>
2  <repository orx_version="1.71">
3
4      <object last_changed="Фамилия И.О." name="DeveloperScript"
        responsible="Фамилия И.О.">
5          <select access_mode="0" is_service="0" last_changed="Фамилия
            И.О." name="DeveloperScript.CriticalErrorFixScript"
            responsible="Фамилия И.О." returns="NONE" type="PYTHON">
6              <definition>
7                  <language>PYTHON</language>
8                  <body>
9                      SqlQuery(""" UPDATE "Таблица1" SET "Поле1" = NULL
10                         """)
11                      result = БизнесОбъект.ОбновитьДанные()
12                      LogMsg(str(result))
13                  </body>
14              </definition>
15          </select>
16      </object>
17 </repository>

```

Пример архива скрипта: s.familiao.io/1174868333_v2.zip.

Например, если скрипт содержит только файл с клиентами prod_cloud_clients, то при выполнении на тестовых стендах скрипт будет выполнен на всех клиентах, а при выполнении на бое – скрипт будет выполнен только на указанных клиентах.

Клиенты из списка могут принадлежать разным продуктам; при запуске скрипта на конкретном продукте, скрипт будет выполнен только на клиентах, принадлежащих этому продукту. В ВНР рекомендуется упомянуть, что скрипт будет выполнен не на всех клиентах, для оптимизации работы эксплуатации.



Указать список сервисов неперсонального приложения, на которых будет выполнен скрипт

Список сервисов, на которых необходимо выполнить скрипт, задаётся только для неперсональных приложений, и является для них обязательным.

В корень архива скрипта, по аналогии с файлами со списком клиентов, необходимо добавить файл, содержащий системные имена целевых сервисов. Для всех стендов файл является общим и именуется "services".

Внутри файла должны содержаться только системные имена сервисов, разделенные пробелами, знаками табуляции или переносами строки. Скрипт будет запущен только на тех сервисах приложения, что указаны в данном файле. Если в составе приложения ни один сервис из файла не был найден, то запуск скрипта завершится соответствующей ошибкой.

Получение файлов модуля

Способ 1.

Имя модуля формируется по следующему правилу: "DeveloperScript_ZipFileName". Т.е. если архив со скриптом называется "MySuperScript.zip", то он развернется в модуль с именем "DeveloperScript_MySuperScript", независимо от того, что задано в s3mod-файле.

```
1 def get_file_path():
2     # имя файла
3     file_name = 'test.csv'
4     # имя архива
5     zip_name = 'MySuperScript'
6     # имя модуля, именно таким оно будет после распаковки на сервере
7     module_name = 'DeveloperScript_' + zip_name
8     # директория, в которой находится csv-файл
9     csv_folder = 'csv_folder'
10    # вычисляем путь
11    base_path = os.path.normpath(os.path.join(ConfigGet('Модули'),
12    module_name, csv_folder))
13    # возвращаем полный путь до файла
14    return os.path.join(base_path, file_name)
```



Рекомендации

Не пытайтесь склеить пути с символом "косая черта" (/), используйте методы, которые учитывают ОС, например, **os.path.normpath**, **os.path.join**. При чтении файлов учитывайте кодировку, например, если csv в 1251, то файл будет прочитан CentOS как UTF8 (кодировка по умолчанию), а это может привести к неожиданным ошибкам.

```
open(get_file_path(), 'r', encoding='cp1251')
```

Способ 2.

Из .ру файла (из кода в .огх этот способ не работает) получить путь до каталога, содержащего этот файл:

```
current_folder = os.path.dirname(os.path.abspath(__file__))
```

Построить путь до любого другого файла в архиве:

```
path_to_file = os.path.join(current_folder, 'file.txt')
```

Выполнить разный код на разных стендах

Если код или данные скрипта специфичны для стенда, где выполняется скрипт, то следует из кода скрипта запросить системное имя текущего стенда методом **CloudEntity.SysName** из модуля **Cloud Interaction** и в зависимости от стенда использовать нужные данные или код.

Список стендов и их псевдонимов находится в сервисе ["Управление облаком"](#): **Настройки** → **Облака**



Рекомендации

Следует использовать данный способ, вместо того, чтобы создавать несколько архивов скрипта под разные стенды.

Пример:

```
1 cloud_sysname = sbis.CloudEntity.SysName()
2
3 if cloud_sysname == 'prod':
4     # выполнится на рабочем
5     ...
6 elif cloud_sysname == 'fix':
7     # выполнится на фиксе
8     ...
9 else:
10    # выполнится на остальных стендах
11    ...
```

Выполнить SQL-запрос на схеме клиента без транзакции. Необходимо для CONCURRENTLY запросов



Побочный эффект sql-инструкции CREATE INDEX CONCURRENTLY

Если в процессе выполнения произошел deadlock или нарушилось ограничение unique, а это произойдет с высокой вероятностью на "живой" БД, то нужно явно вызвать DROP INDEX [CONCURRENTLY], иначе создаваемый индекс останется в базе в неактуальном состоянии (так как операция происходит вне транзакции), что в последствии приведет к проблемам конвертации.

```
1 scheme = "_" + sbis.Session.ID()[ :8]
2 drop_index_sql = '''DROP INDEX CONCURRENTLY IF EXISTS
3 {}."index1"''' .format(scheme)
4
5 create_index_sql = '''CREATE INDEX CONCURRENTLY "index1" ON "
6 {}"."Table1" ("Column1", "Column2")''' .format(scheme)
7
8 statement = sbis.GetConnectedDatabase().CreateStatement(False)
9
10 # Удалить индекс, если он существовал.
11 statement.Exec(drop_index_sql)
12 try:
13     # Накатить индекс.
14     statement.Exec(create_index_sql)
15 except sbis.BadExecStatement:
16     # В случае ошибки удалить индекс, чтобы он не остался в
17     неактуальном состоянии.
18     statement.Exec(drop_index_sql)
19     raise
```

Если требуется использовать в запросе схему public, например для создания индекса с использованием типа hstore, то перед вызовом нужно установить параметры запроса. Но делать через pgbouncer так нельзя, потому что запрос может уйти в другое соединение. Поэтому требуется сделать прямое соединение с БД самостоятельно.

```
1 import sbis
```

```

2 import json
3
4 db_config =
    json.loads(sbis.ConfigGet('clusters.direct_db_connections'))
5 for item in db_config:
6     # Создаем подключение
7     cstring = item['master']
8     db = sbis.CreateDatabase(cstring)
9     statement = db.CreateStatement(False)
10    scheme = "_" + sbis.Session.ID()[ :8]
11    # установка схемы клиента и схемы public
12    statement.Exec('SET search_path TO {}, public;'.format(scheme))
13    statement.Exec('' select 1 ''')
14    # Так как подключение создали мы сами, закрывать его нужно тоже
    самим
15    db.Close()

```

 Индексы, добавляемые через VHR, будут удалены конвертером при следующей конвертации (при отсутствии в discx).

Выполнить скрипты по всем БД своего сервиса

Если у сервиса несколько БД и нужно выполнить скрипт на всех базах, то можно использовать следующий подход:

- Основной метод скрипта: **DeveloperScript.Script()**:

```

1 import sbis
2 import json
3 # получение всех баз сервиса
4 db_config = json.loads(sbis.ConfigGet('clusters.databases'))
5 futures = []
6 for db in db_config:
7     # параллельный вызов метода со скриптом на каждой БД
8     x = db['id']
9     future =
        sbis.BLObject("Invoker").FutureInvoke("ExecuteOnDatabase", db['id'])
10    futures.append(future)
11    # ждем выполнения всех вызовов
12 for future in futures:
13    future.get()

```

- Создадим бизнес-объект с методом **Invoker.ExecuteOnDatabase(db_id)**, вызываемым из основного скрипта:

```

1 # устанавливаем нужную БД
2 sbis.SelectCurrentDB(db_id)
3 # собственно скрипт для БД
4 SqlQuery('' sql-код ''')

```

Подробнее описано в статье [Эффективная работа с удаленными запросами. Shared future](#).

Выполнить скрипт на схеме public

При выполнении скрипта на мультиарендных (персональных) сервисах существует возможность выполнения дополнительного кода только на схеме public. Для этого нужно в справочнике ogh описать методы:

- DeveloperScript.BeforeScript()** – метод выполняется до выполнения методов на схемах клиентов.
- DeveloperScript.AfterScript()** – метод выполняется после выполнения методов на схемах клиентов.

Возможно описать только один из методов. Методы выполняются в транзакции, с установкой search_path = public.

Ограничение одновременного выполнения клиентских скриптов



Важно!

Разработчик несет полную ответственность за нагрузку, порождаемую созданным скриптом, и должен четко понимать, какие действия выполняет скрипт.

Также следует осознавать, какой сервис или какая подсистема будет являться узким местом при выполнении скрипта и примерно оценить ее пропускную способность в тех операциях, которые выполняет скрипт.

Помимо вызовов на сервисы, скрипт может оказывать влияние на систему другим образом, например, создавать сценарии dws.

Если это так, то необходимо оценить число таких сценариев и обеспечить такую политику исполнения скрипта, чтобы число одновременно создаваемых скриптов не превышало 300000.

В противном случае, разделить исполнение скрипта на несколько этапов за несколько дней.

По умолчанию любой скрипт одновременно выполняется на каждой БД на одном клиенте, то есть максимальное число одновременно выполняющихся методов скрипта равно общему числу баз данных всех сервисов, на которых выполняется скрипт.

Такой сценарий хорошо подходит для SQL-скриптов, действующих в рамках схемы клиента, но не подходит для скриптов, выполняющих внешние (особенно асинхронные) вызовы на третьи сервисы, так как велик риск создать очередь на них.

Чтобы избежать этого, добавлена возможность ограничивать число одновременно выполняющихся методов скрипта. Задать ограничение можно в ВНР в поле **Ограничение одновременного выполнения скриптов**.

до 30.06.24

Исполнитель

Релиз-менеджмент × Укажите исполнителя или [участок работ](#)

Описание

▼

Что нужно выполнить и срок

Когда выполнить *

Срочность *

Тип работы *

Приложение

Режим запуска

Уточнение

Выполнить на dev *

Инструкция

Ограничение одновременного выполнен...

Время выполнения скрипта *

Можно останавливать и перезапускат... *

В случае ошибок при выполнении на б... *

Выполнить работы в облаке *

Решает проблему *

Укажите когда нужно выполнить работу

Не авария. Выполнить в штатном режиме (23:00-06:00)

Скрипт ⓘ

Все приложения online ⓘ

Копия сервиса ⓘ

Дополните информацию при необходимости ⓘ

Не выбрано

Укажите число одновременно выполняющихся методов скрипта ⓘ

Для основного сервиса - время выполнения скрипта на одной ⓘ

Не выбрано

Не выбрано

☐ RU
☐ KZ
☐ UZ

Укажите причину выполнения/ссылку на ошибку/ссылку на ⓘ

Дополнительные сведения

Простой в работе сервиса *

Ожидаемые проблемы при выполнении

Как проверить

Нужна доброска *

Инструкция по откату

Синхронизировать демо шаблоны

Не выбрано

Укажите возможные проблемы в работе облака или их ⓘ

Укажите сценарии для проверки ⓘ

Не выбрано

Укажите последовательность действий для отката ⓘ

☒ Нет ⓘ

Например, если задано ограничение 100, то скрипт будет выполняться не более, чем на 100 клиентах одновременно. Ограничение работает на общее количество одновременно выполняющихся методов скрипта на всех приложениях. По умолчанию данное ограничение не задано.

Диапазон значений: от 0 до 999, где 0 - выполнение не ограничено, 999 - максимально возможное значение ограничения.



В случае использования ограничения

Задание небольшого ограничения может сильно замедлить выполнение скрипта на всех клиентах, поэтому пользоваться этим надо только тогда, когда это действительно необходимо, и оценивать ожидаемое время выполнения скрипта на всех клиентах.

Число потоков сервиса

По умолчанию скриптовая БЛ для неперсональных сервисов запускается с двумя рабочими процессами. Этого вполне достаточно для большинства случаев: один процесс используется для исполнения метода скрипта, второй – для возможных асинхронных вызовов на самого себя.

Некоторые разработчики организуют таким образом скрипт, что он порождает множество асинхронных запросов на самого себя, тем самым распараллеливая свою работу. С двумя рабочими процессами все такие запросы выстроятся в очередь и будут исполняться последовательно.

Разработчик может указать в ВНР, что для скрипта требуется большее число рабочих процессов. Сотрудник службы эксплуатации при этом должен это значение задать на диалоге запуска скрипта.

Минимальное значение – 2, максимальное – ограничено параметром **Управление версиями.Скрипты.МаксимальноеЧислоПроцессовСкриптовыхБЛ**.

Частичное и повторное выполнение скриптов на клиентах

Если скрипт не был выполнен до конца на всех клиентах (например скрипт прервали по некоторой причине), то новый запуск этого же скрипта на этом же приложении продолжит выполнение на еще не обработанных клиентах, а не на всех как раньше.

Файловые логи и данные в Redis, опубликованные через API, выгружаются в хранилище файлов в момент завершения запуска скрипта, привязываются к этому выполнению и прикрепляются к отчету скрипта. Если скрипт выполняется за несколько ночей, то будет столько записей в отчете – сколько раз был запущен скрипт, и в каждой записи будут логи и файлы относящиеся к соответствующему запуску скрипта.

При необходимости возможен полный повторный запуск на всех клиентах. Если скрипт был выполнен на всех клиентах приложения, то в случае повторного запуска скрипт будет выполнен заново на всех клиентах. Под всеми клиентами подразумеваются либо все клиенты всех баз приложения, либо конкретные клиенты, указанные в архиве скрипта.

Уникальность скрипта определяется по хэш-сумме архива скрипта, если изменить содержимое архива (включая изменение файлов скрипта, числа файлов скрипта, пересоздание архива, но не включая изменение названия архива) – это будет считаться новым скриптом.

Как проверить скрипт локально

Чтобы проверить скрипт локально, достаточно:

1. Поместить содержимое подготовленного архива в папку с исходными модулями вашего сервиса и выполнить Deploy сервиса. Также можно поместить содержимое архива в папку с модулями бизнес-логики уже в развернутом сервисе.
2. Если на локальном стенде используются права, выдать права на добавленный метод в используемую роль.
3. Запустить сервис и вызвать описанный метод модуля DeveloperScript любым способом.
4. Проверить результат выполнения.